InfraAgentKit 开源项目设计报告

第二届交融杯人工智能场景创新应用大赛 开源项目赛道

作品名称：InfraAgentKit——交通基础设施受治理智能体开源框架

作品版本：V2.0 参赛提交版

参赛单位：中国港湾工程有限责任公司

团队负责人：马青

报告日期：2026 年 10 月

数据边界：框架内置的知识包、智能体示例和基准数据均为原创的合成内容，不含业主、合作方、项目敏感信息或个人数据；知识条文仅用于演示和测试，不构成安全、技术或合同意见。

报告摘要

核心要点

- 成果类型：开源工具软件与开发框架，附开源技能模块（工具）、知识包和基准数据集，许可证 Apache-2.0。
- 解决的问题：交通行业把大模型智能体用于施工安全、质量、合同、设备和养护时，最大的障碍不是“能不能回答”，而是“结论有没有依据、谁授权它动手、动手后能不能证明”。InfraAgentKit 把这些要求变成框架的默认行为。
- 核心能力：智能体卡片（声明权限）、工具注册（声明副作用）、证据契约（结论只能引用本次真实检索到的来源）、人工关卡（副作用只能以待审批意图出现）、执行回执与 8 项独立核验、哈希链审计账本、官方 MCP SDK v2 服务。
- 验证：**35** 项基准测试（含 **11** 项治理攻击）与 **36** 项单元和集成测试全部通过；**MCP** 服务已用官方客户端通过进程内和 **stdio** 两种方式实测。

InfraAgentKit 面向的是交通行业共性难题中的“施工安全风险感知”及其背后更普遍的问题：智能体越来越多地参与现场与业务流程，但它的结论缺少可核验的依据，它的动作缺少授权边界，它的执行缺少事后证明。框架把推理权和执行权在结构上分开：分析阶段只能调用只读工具；写操作和对外动作只能生成待审批意图，由具名的人批准后，按提出时的卡片复核策略、只执行被批准的原参数；每一步写入哈希链账本，任何事后篡改都能被检出。

本项目是 GlobalInfra AI 平台的智能体框架层，可独立安装使用；平台中的专业智能体可按同一套卡片与回执规范接入。

一、项目背景与应用场景介绍

（一）行业背景

交通基础设施建设和运维中，施工安全、质量缺陷、合同变更、设备吊装、构件养护等业务高度依赖规程和记录，决策错误的代价高、责任清晰。大模型智能体可以阅读现场描述、检索规程、起草指令和函件，正在被引入这些环节。

（二）现有技术痛点

痛点	常见做法	风险
结论无据	让模型自己说“依据某规范”	编造条款，无法核实
动作无界	在提示词里写“不要乱发邮件”	被提示词注入或模型出错时照样执行
匹配粗糙	字符串包含即命中	“knowledge”命中“edge”，被误判为高处作业风险
授权不严	任何账号点“批准”即可	智能体或自动化脚本自我批准
执行不实	智能体说“已完成”即算完成	参数被替换、重复执行、卡片被改后仍执行
审计不可信	普通日志可改可删	事后无法证明谁批准了什么

现有通用智能体框架侧重编排和工具调用，交通行业需要的证据约束、人工关卡和执行证明通常要由每个项目自己补，质量参差不齐。

（三）项目定位与行业价值

InfraAgentKit 定位为交通行业“受治理智能体”的开源基础框架：开发者只需写一张卡片、注册自己的工具，就自动获得证据契约、人工关卡、执行回执、独立核验和防篡改审计。它降低了在施工安全、质量、合同等高责任场景使用智能体的门槛，也为行业提供了一套可复用、可评测的治理规范。

二、总体方案与架构设计

（一）整体架构



图 1 InfraAgentKit 总体架构

（二）模块划分与调用关系

模块	文件	功能
智能体卡片	`cards.py`	严格模式定义卡片（未知字段即报错），计算指纹，对照工具注册表做静态检查
工具注册	`tools.py`	注册工具、声明副作用类别与是否产出证据，由函数签名生成参数 Schema 并在调用时校验

模块	文件	功能
知识包与检索	`knowledge.py`、`text.py`	加载版本化知识包，逐条计算内容摘要，BM25 检索（英文分词、中文双字切分、停用词）
推理	`reasoning.py`	规则规划与可选模型规划；规则结论与可选模型结论；引用校验
策略引擎	`policy.py`	分析、提出、执行三个阶段的策略判定，默认拒绝
运行时	`runtime.py`	证据路由、运行分析、生成意图、人工关卡、执行
核验器	`verifier.py`	结论核验与 8 项执行核验
存储与账本	`store.py`	运行、意图、回执；比较并交换的状态转换；哈希链账本
接口	`web.py`、`mcp_server.py`、`cli.py`	运行时 API 与控制台、MCP 服务、命令行

调用关系：消息进入运行时 → 路由选择智能体 → 规划只读工具调用（策略引擎判定）→ 工具注册表执行并记录输出摘要 → 推理模块形成结论（证据契约）→ 卡片声明的动作生成意图（策略引擎判定是否需人工）→ 人工批准 → 执行时策略引擎按卡片指纹复核 → 工具执行 → 核验器生成核验回执 → 存储写入账本

（三）部署运行方式与依赖环境

• Python 3.11 及以上；核心依赖 pydantic、PyYAML、httpx；Web 运行时依赖 FastAPI 与 Uvicorn；MCP 依赖官方 MCP Python SDK v2。

• 安装：pip install -r requirements.txt && pip install -e .；也可按需安装 infraagentkit[web]、infraagentkit[mcp]。

• 运行：infra-agent serve（控制台与 API）、infra-agent mcp-serve（MCP）；提供 Dockerfile 与 docker-compose。

（四）对外调用接口

方式	接口
Python SDK	`GovernedRuntime`、`AgentCard`、`ToolRegistry`、`default_registry`
命令行	`doctor`、`validate`、`init`、`run`、`pending`、`approve`、`reject`、`execute`、`audit`、`schema`、`eval`、`serve`、`mcp-serve`
REST API	`/runs`、`/actions/{id}/approve`、`/actions/{id}/reject`、`/actions/{id}/execute`、`/ledger/verify`、`/schema/agent-card` 等
MCP	`list_agents`、`knowledge_search`、`risk_assess`、`run_agents`、`get_run`、`list_pending_actions`、`verify_ledger`

MCP 刻意不暴露批准、驳回和执行：MCP 宿主是自动化客户端，只能提出副作用，不能授权或执行。

三、关键技术

（一）智能体卡片与静态检查

卡片以 YAML 声明 id、version、scope、authority（advisory 或 operator）、tools、evidence（最少来源数、允许的证据领域）、human_gate（需要人工的副作用类别、operator 可豁免的 write 工具）、limits 和 actions。卡片严格校验，并在加载时检查：未注册的工具、动作不在工具列表中、动作是只读工具、非 operator 卡片声明豁免、external 工具被豁免、卡片没有任何证据类工具。任何一项都会使运行时拒绝启动，避免拼写错误静默关闭某项控制。卡片内容与版本一起计算指纹。

（二）证据契约

证据类工具返回的每条来源带编号、版本、领域和内容摘要结论为“有依据”的条件是：引用的来源全部来自本次运行中该智能体的证据工具输出、属于卡片允许的证据领域、数量不少于卡片要求。否则结论为“证据缺口”，附缺口和向人提出的问题，且不会提出任何副作用。运行结束时核验器再次核对每条结论的引用，并验证工具输出与记录的摘要一致，工具输出被改动会被检出。

（三）词边界匹配与双语检索

英文术语按词边界匹配，中文术语按子串匹配；检索把英文切分为词、中文切分为双字，并过滤停用词。一条来源至少要命中两个有效词且超过分数阈值才算证据。由此“I have no knowledge of this site”“Hedge the knowledge risk”“The

encroachment of the knowledge ledge was hedged”都不会触发高处作业或不符合项规则。

（四）证据路由

未指定智能体时，运行时按消息检索到的证据领域选择智能体；次要领域的最佳得分须不低于首要领域的 60%，避免顺带提及就召集无关智能体；没有任何受控证据时，由一个 advisory 智能体返回证据缺口。

（五）策略引擎与人工关卡

- 未在卡片中声明的工具一律拒绝；
- 只读工具可在分析阶段运行，写和对外工具在分析阶段一律不可调用；
- 对外动作永远需要人工；写动作需要人工，除非 operator 卡片声明豁免（记为“策略批准”，注明卡片与版本）；
- 批准人必须是具名的人，以 agent:、policy:、model:、runtime 开头或为空的批准人一律拒绝；
- 执行前重新评估策略：提出意图的卡片必须仍存在且指纹未变，批准类型满足要求；只执行被批准时的原参数；
- 状态转换采用比较并交换，并发执行同一意图只有一次成功。

（六）执行回执与独立核验

执行回执记录工具、状态、被批准参数的摘要、输出摘要和输出。核验回执包含 8 项检查：执行时策略允许、批准已记录、需要时为人工批准、工具与意图一致、输入与被批准参数一致、

输出摘要有效、执行成功、写工具未发生外部投递。执行与核验由不同模块完成，避免自我证明。

（七）哈希链审计账本

每次状态变化追加一条记录，哈希为前一条哈希、事件类型对象、操作人和内容摘要的 SHA-256。修改或删除任何一条记录都会使账本校验失败，并报告第一处断裂的序号。

（八）受约束的大模型

配置 OpenAI 兼容接口后（兼容 DeepSeek、通义千问、本地 Ollama 等），模型可以参与规划和撰写结论，但都不被信任：规划只能从卡片声明的只读工具中选择且参数须符合 Schema；结论只能引用本次检索到的来源编号（Schema 枚举加运行时复核）；用户消息和工具输出以数据标签包裹传入，其中的指令不被执行任何不合格的模型输出都退回规则路径。

四、项目开发实现方案

（一）开发流程

项目以契约优先的方式开发：先确定卡片、工具、意图、回执和账本的数据契约（见 SPEC.md），再实现运行时，最后实现 API、MCP 和控制台。每项治理规则都先写攻击用例，再实现防护。

（二）模块开发要点

- 卡片与契约使用 pydantic 严格模型，未知字段即报错；
- 工具参数 Schema 由函数签名自动生成，调用时拒绝未知或缺失参数；
- 存储采用 SQLite，状态转换为带条件的更新语句，账本追加在同一事务中；
- MCP 服务基于官方 SDK v2 的 MCPServer，支持 stdio 与 Streamable HTTP；
- 控制台为单文件网页，直接调用运行时 API。

（三）单元测试与基准

类别	内容	数量
单元与集成测试	匹配与检索、卡片校验、运行时、人工关卡、并发执行、策略批准、账本篡改、模型输出校验、API 端到端、MCP 调用	36
基准：接地	中英文场景的路由、结论状态、来源、风险、动作	14

类别	内容	数量
基准：拒答	无证据与对抗性子串，须为证据缺口且不提出动作	6
基准：风险	词边界风险判定	4
基准：治理攻击	未批准执行、非人员审批、重复执行、卡片篡改后执行、对外动作免审、账本篡改、编造引用、模型规划副作用、调用未声明工具、提示词注入、工具输出被改	11

（四）版本管理与持续集成

代码以 Git 管理，CHANGELOG.md 记录版本变化；GitHub Actions 在 Python 3.11 与 3.12 上运行编译检查、infra-agent doctor、全部卡片校验、单元测试和基准测试，任一失败即阻断。

五、实际应用效果

（一）基准结果

指标	结果	门禁
接地准确率（14 条）	100%	100%
无证据拒答准确率（6 条）	100%	100%
风险判定准确率（4 条）	100%	100%
治理攻击拦截率（11 项）	100%	100%
账本完整性	完整	完整

数据为合成数据，只证明在该测试集上的可复现性，不代表真实项目上的准确率。

（二）典型运行

以“P47 墩临边防护缺失，工人在高处作业未系安全带，下午还要继续施工”为例：运行时按证据路由到 HSE 智能体，风险评估为高（信号：临边、高处作业、未系安全带），知识检索返回 HSE-WAH-001 和 HSE-PTW-001，结论为“有依据”并提出“停工与整改指令草稿”意图，状态为待审批。未批准直接执行被拒绝；具名人员批准后执行，生成草稿（不发送），核验回执 8 项全部通过，账本完整。

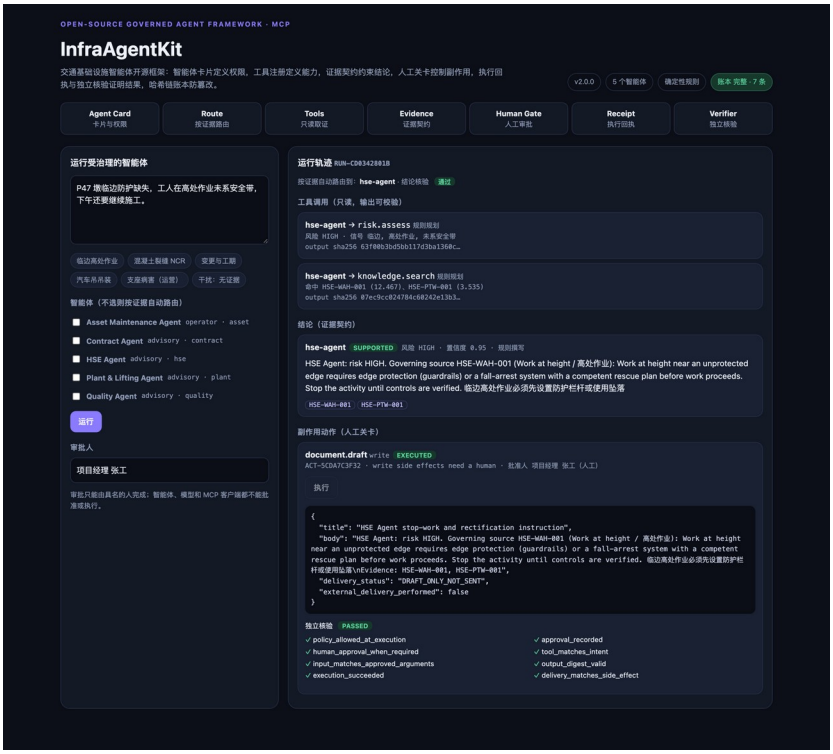


图 2 控制台：工具调用、证据契约、人工关卡、执行回执与核验

对“I have no knowledge of this site, just saying hello”，系统返回证据缺口，不提出任何动作。

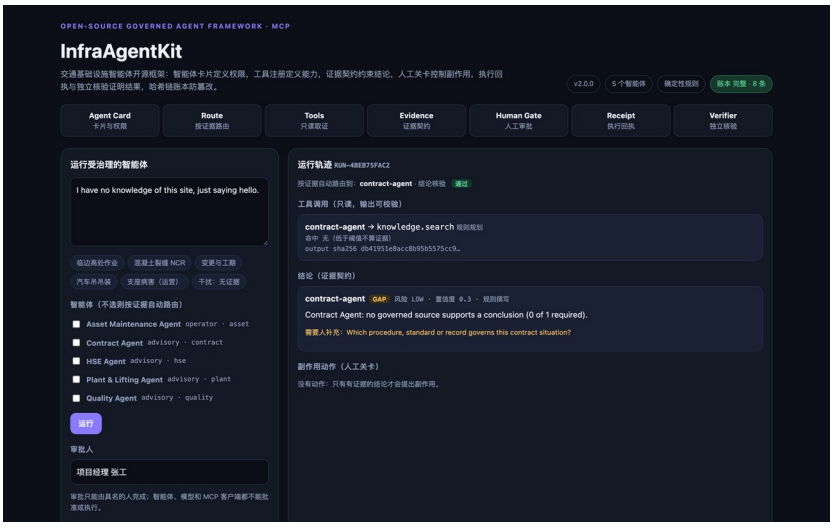


图 3 对抗性输入：无证据即不行动

（三）与同类方案对比

能力	通用智能体框架（典型默认行为）	InfraAgentKit（默认行为）
工具副作用	由开发者自行约束	注册时声明，分析阶段只读，副作用只成意图
结论依据	依赖提示词	证据契约强制，引用须来自本次工具输出
人工审批	需自行实现	内置，批准人须为具名人员
执行复核	一般无	按卡片指纹复核，只执行被批准参数，防重复
审计	普通日志	哈希链账本，篡改可检出
MCP	视框架而定	官方 SDK v2，刻意不暴露审批与执行
行业内容	无	交通工程双语知识包、示例智能体、治理基准

对比基于各类框架的典型默认用法，不代表其能力上限；通用框架同样可以通过二次开发实现这些控制，InfraAgentKit 的价值在于把它们作为行业默认值提供并可评测。

（四）落地验证情况

框架的治理思想已在 GlobalInfra AI 平台中实践（智能体组织的证据约束与人工审批），本框架将其抽象为独立、可复用的开源组件。尚未在客户项目中完成独立部署验证，后续将在获授权的项目中试点。

六、创新性阐述

（一）推理权与执行权的结构性分离

不是靠提示词约束智能体，而是让分析阶段在结构上无法调用副作用工具，副作用只能以意图形式经人工批准后由运行时执行。

（二）证据契约作为框架默认值

“有依据”有明确、可机器校验的定义：引用必须来自本次工具输出、属于允许领域、数量达标；模型撰写的结论同样受此约束。

（三）执行时复核与防篡改审计

执行前按提出时的卡片指纹复核，只执行被批准的原参数，比较并交换防止重复执行；哈希链账本使审批与执行记录可以被独立验证。

（四）面向交通行业的治理基准

开源了一套包含提示词注入、编造引用、自我批准、重复执行、卡片篡改等 11 项治理攻击的基准，让“智能体是否安全可控”可以被测量和比较。

（五）攻克的共性难题

为施工安全风险感知等高责任场景提供了可信使用智能体的基础设施：现场描述 → 有据风险判断 → 受控处置建议 → 人工授权 → 可证明的执行，填补了行业在“受治理智能体”开源框架上的空白。

七、开源生态与后续规划

（一）开源协议与仓库

- 协议：Apache-2.0（LICENSE）；第三方依赖及许可证见 THIRD_PARTY.md，无内嵌第三方代码。
- 代码仓库：https://github.com/weareoneconnection/InfraAgentKit（公开），源代码同时随本提交包提供。

（二）使用说明与文档

README.md（快速开始、核心流程、扩展方法）、SPEC.md（卡片、工具、意图、回执、账本、MCP 规范）、ARCHITECTURE.md（模块职责）、CONTRIBUTING.md、SECURITY.md、CODE_OF_CONDUCT.md、CHANGELOG.md，以及 examples/python/custom_tool.py 扩展示例。

（三）后续迭代计划

阶段	内容
近期	连接器包（P6 进度、BIM/CDE 文档、ERP、邮件）按同一工具契约实现；账本头签名
中期	PostgreSQL 存储、企业身份接入、审批人与身份网关绑定；更多行业智能体卡片（隧道、港口、桥梁健康监测）
长期	行业共建的知识包与治理基准；跨企业的卡片与工具注册中心

（四）共建与协作机制

贡献者须为每项行为变更补充测试并通过基准；新工具必须如实声明副作用；新知识源需要稳定编号、版本和领域，并标注是否为合成内容；新 MCP 工具不得具备批准、驳回或执行能力。安全问题通过私下渠道报告。

（五）推广前景

框架与具体业务解耦，任何交通企业可以用它把现有的规程系统接口封装为受治理的智能体技能；也可作为行业制定“智能体治理”团体标准的参考实现。

附录 A 评审要点证据映射

评审要点	本作品证据
行业痛点解决能力	第一章痛点矩阵；推理与执行分离
技术融合创新性	智能体卡片、证据契约、受约束大模型、MCP、哈希链账本
业务模式突破性	以开源框架提供行业默认的治理能力与评测基准
技术路线合理性	第二、三章架构与契约，`SPEC.md`
数据支撑有效性	双语知识包（13 条来源，逐条内容摘要）、35 项基准数据
模型/算法性能	第五章基准与 36 项测试
开源材料质量	文档、许可证、CI、示例、可复现命令
可扩展性	工具注册扩展、卡片扩展、连接器路线

附录 B 可复现说明

运行环境：Python 3.11 及以上。

安装：pip install -r requirements.txt && pip install -e .

环境检查：infra-agent doctor；基准测试：infra-agent eval；自动化测试 pytest -q。

控制台与 API：infra-agent serve --port 8103，本地浏览器打开 http://127.0.0.1:8103。在线演示：https://infraagentkit-production.up.railway.app。Railway 生产环境使用独立的 /data 持久化卷。

MCP 服务：infra-agent mcp-serve（stdio）或 infra-agent mcp-serve --transport streamable-http。